BFS → ne parcurge tot graful

```
BFS(source, G=(nodes, adj))
    FOR node ∈ nodes
        c[node] = alb
        d[node] = ∞
        p[node] = NULL

    c[source] = gri
    d[source] = 0
    //p[source] = NULL

    queue = new Queue(..)
    queue.push(source)

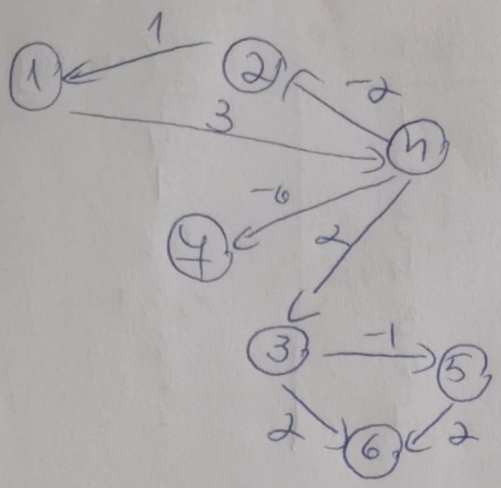
    WHILE !queue.isEmpty()
        node = queue.pop()
        FOR neigh ∈ adj[node]
            if c[neigh] == alb
                c[neigh] = gri
                d[neigh] = d[node] + 1
                p[neigh] = node
                queue.push(neigh)
        c[node] = negru
```

COMPLEXITATE

- lista de adiacență $O(|V|+|E|)$
- matrice — " — : $O(|V|^2)$

(1)

Sl. 2 2020)

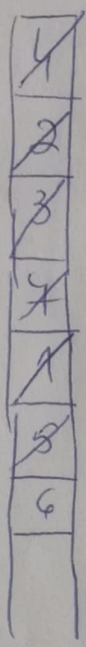


node	adj[node]
1	4
2	1
3	5,6
4	2,3,7
5	6
6	-
7	-

BFS din 4 + ordine parcurgere

(2)

queue



ator node neigh

(1): 4 → 2, 3, 7

(2): 2 → 1

(3): 3 → 5, 6

(4): 7 → -

(5): 1 → - (le adaug numai pe cele care nu au mai fost deloc descoperite)

(6): 5 → -

(7): 6 → -

node	1	2	3	4	5	6	7
p[node]	2	4	4	NULL	3	3	4

node	1	2	3	4	5	6	7
d[node]	2	1	1	0	2	2	1

TEOREMĂ

Fie (∀) nod $v \in V$ oarecare din sursa s

$d[v] = \delta[s][v]$ (cea mai mică distanță de la s la v)

⇒ Nu contează ordinea în care sunt parcurși vecinii nodurilor din listele de adiacență

DFS → parcurge tot graful

```
DFS(G = (nodes, adj))
    FOR node ∈ nodes
        p[node] = NULL
        c[node] = alb
        d[node] = 0 // timp de descoperire
        f[node] = 0 // timp de finalizare
    time = 0
    FOR node ∈ nodes
        IF c[node] == alb
            DFS_helper(node, G)
```

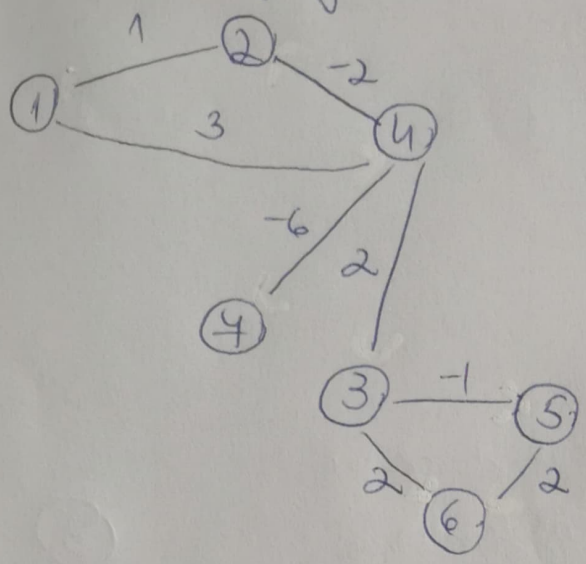
(3)

COMPLEXITATE

- lista de adiacență: ~~Θ~~$(|V|+|E|)$
- matrice -11-: ~~Θ~~$(|V|^2)$

```
DFS_helper(node, G = (nodes, adj))
    time++
    d[node] = time; c[node] = gri
    FOR neigh ∈ adj[node]
        IF c[neigh] == alb
            p[neigh] = node
            DFS_helper(neigh, G)
    c[node] = black
    time++
    f[node] = time
```

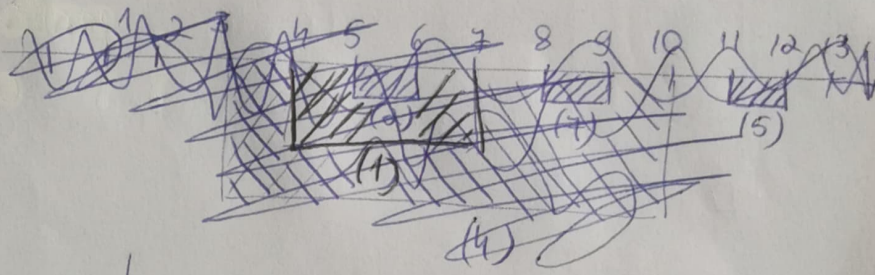
S1.1 2020) graful neorientat



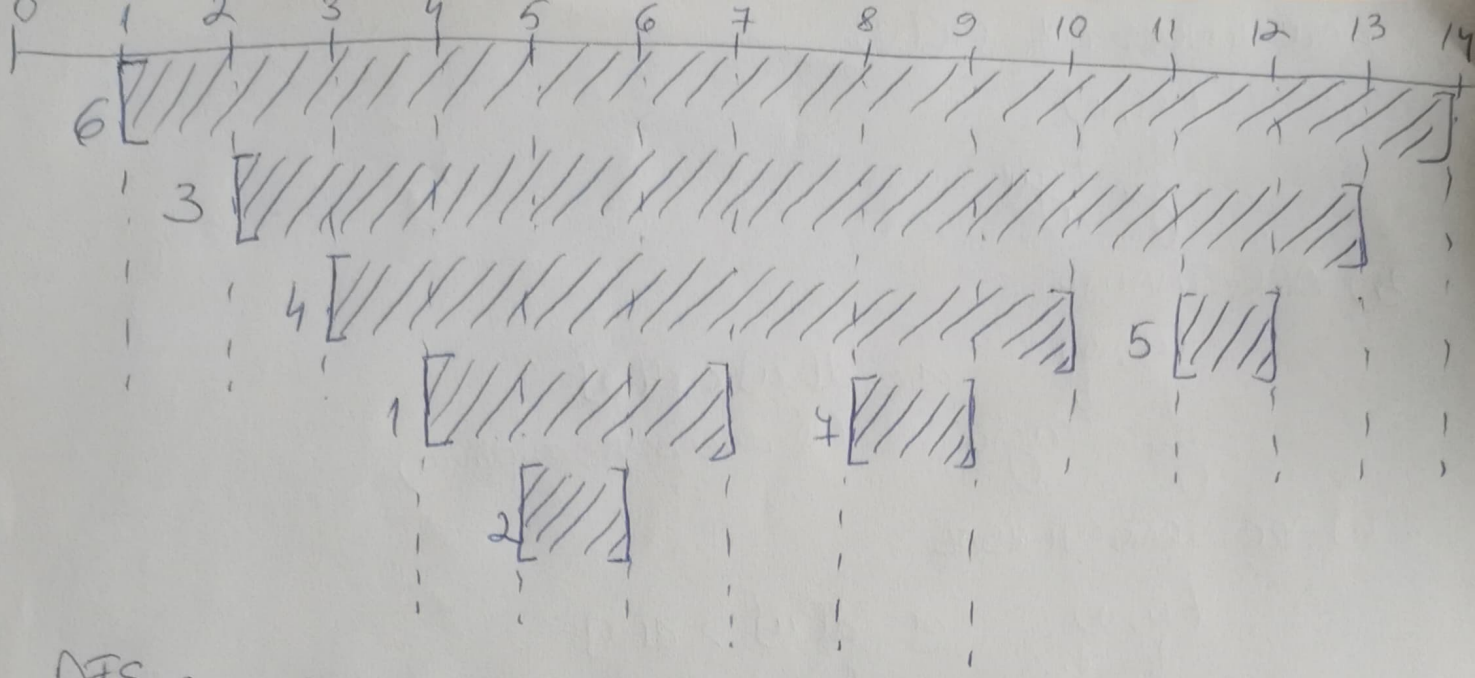
node	adj[node]
1	2, 4
2	1, 4
3	4, 5, 6
4	1, 2, 3, 7
5	3, 6
6	3, 5
7	4

DFS din 6 + timpi start, finish

(4)



DFS(6)
d[6] = 1
DFS(3)
d[3] = 2
DFS(4)
d[4] = 3
DFS(1)
d[1] = 4
DFS(2)
d[2] = 5
f[2] = 6
f[1] = 7
DFS(7)
d[7] = 8
f[7] = 9
f[4] = 10
(*)
(*) DFS(5)
d[5] = 11
f[5] = 12
f[3] = 13
f[6] = 14



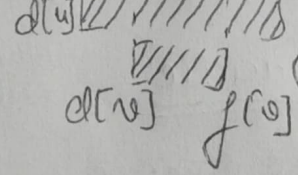
DFS generează câte un arbore de acoperire pt. fiecare componentă conexă a grafului. → pădure de arbori

(ordinea în care se parcurg vecinii contează ⇒ arbori diferiți)

TEOREMA PARANTEZELOR

→ în arborele generat de DFS

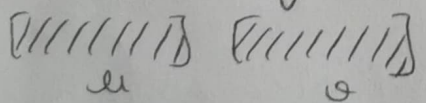
Dacă v este un descendent al lui u atunci v este descoperit cât timp u este explorat ($d[u] < d[v] < f[v] < f[u]$)



(5)

3 CAZURI:

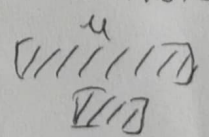
INTERVALE DISJUNCTE



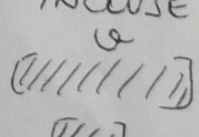
u nu este descendent al lui v

+ v —11— u

INTERVALE INCLUSE



v descendent al lui u



u descendent al lui v

TIPURI DE MUCHII

1) ARC DIRECT (DE ARBORE)

(u, v)

↓ ↓

gri alb

2) ARC INVERS (DE CICLU)

(u, v)
↓ ↓
gri gri

3) ARC ÎNAINTE

(u, v) și d[u] < d[v]
↓ ↓
gri negru (v descoperit după u)

4) ARC TRANSVERSAL

(u, v) și d[u] > d[v]
↓ ↓
gri negru (u descoperit după v)

PROPRIETĂȚI

1) G aciclic dacă nu găsim arce inverse în timpul execuției DFS
2) Într-un graf neorientat DFS poate găsi doar arce directe și inverse

SORTARE TOPOLOGICĂ

G = (V, E) orientat aciclic → dacă are forma (u) ⇄ (v)
atunci u trebuie să se facă înaintea lui v și v înaintea lui u (imposibil)

Trebuie să găsim o ordonare a nodurilor a.î.

dacă (u, v) ∈ E atunci $index_u < index_v$

(u trebuie să apară în soluție înaintea lui v)

1) REALIZARE cu DFS

→ rulăm DFS și alegem nodurile în ordinea descrescătoare a timpului de finalizare

```
TOPSORT (G=(nodes, adj))
    FOR node ∈ nodes
        visited[node] = false
    sort = []
    FOR node ∈ nodes
        IF ! visited[node]
            DFS (node, G, visited, sort)
    return sort.reverse()
```

```
DFS (node, G, visited, sort)
    visited[node] = true
    FOR neigh ∈ adj[node]
        IF ! visited[neigh]
            DFS (neigh, G, visited, sort)
    sort.push (node) // adaugă nodurile în ordinea crescătoare a
                     // timpului de finalizare => trebuie inversate
                     la final
```

(4)

COMPLEXITATE (TEMPORALĂ)

$O(|E|+|V|)$ de la DFS

2) Rezolvare cu BFS (Kahn)

Se bazează pe următoarea proprietate:

Orice graf orientat aciclic are cel puțin un nod cu în-degree == 0 și cel puțin un nod cu out-degree == 0

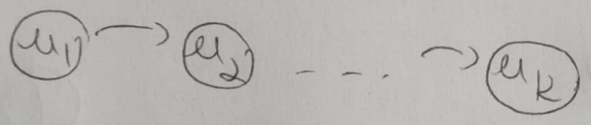
(in-degree(u) = câte muchii intră în nodul u
out-degree(u) = câte muchii ies din nodul u)

DEMONSTRAȚIE

Știm că G este aciclic ⇒ toate căile au lungime finită.

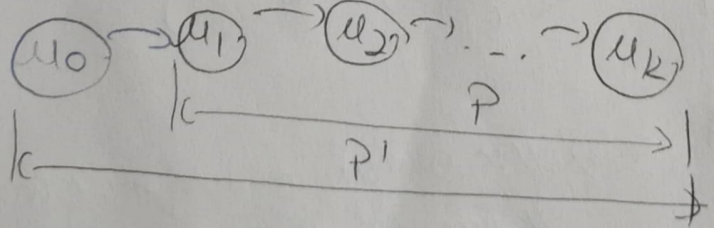
Fie P cea mai lungă cale din G.

⇒ (∃) $u_1, \dots, u_k$ a.î. $P = [u_1, u_2 \dots, u_k]$



Atunci { in-degree(u_1) = 0
out-degree(u_k) = 0 (8)

Dacă in-degree(u_1) ≠ 0 atunci (∃) u_0 a.î. $(u_0, u_1) \in E$



Am găsit P' cale mai lungă decât P

⇒ contradicție (P trebuia să fie cea mai lungă)

⇒ Obligatoriu { in-degree(u_1) = 0
out-degree(u_k) = 0

⇒ (∀) G graf orientat aciclic, (∃) cel puțin un nod cu în-degree == 0 și cel puțin unul cu out-degree == 0

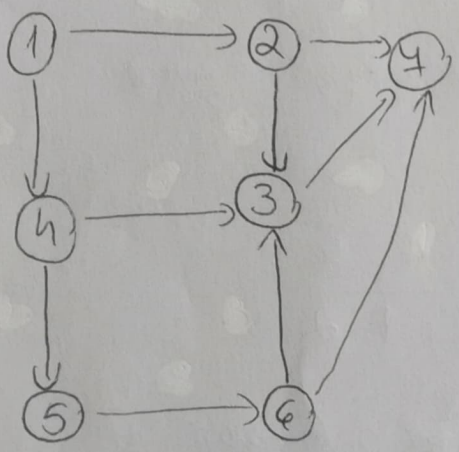
ALGORITM Kahn

COMPLEXITATE: $O(|V|+|E|)$

1) Calculez in-degree pt. (V) noduri.
2) Creez o coadă unde adaug toate nodurile care au in-degree == 0
3) BFS (cât timp am noduri în coadă (=) cât timp (∃) noduri cu in-degree == 0)
   - 3.1) Scot un nod
   - 3.2) Măresc nr. nodurilor vizitate (util pt. verificare dacă (∃) cicluri)
   - 3.3) Pentru fiecare vecin merg și scad in-degree[merg] cu 1 (echivalent cu ștergerea muchiei (nod, merg))
   - 3.4) Dacă merg are in-degree == 0 atunci se adaugă în coadă
4) Dacă nr. noduri vizitate ≠ nr. noduri => (∃) ciclu
   Altfel, întoarce sortarea topologică.

(9)

S1.2 2017CC)



sortare topologică a grafului

node	adj[node]
1	2,4
2	3,7
3	7
4	3,5
5	6
6	3,7
7	-

1) DFS(1): timp start 1 → 14 timp finish

DFS(2): 2 → 7

DFS(3): 3 → 6

DFS(7): 4 → 5

DFS(4): 8 → 13

DFS(5): 9 → 12

DFS(6): 10 → 11

node	1	2	3	4	5	6	7
f[node]	14	7	6	13	12	11	5

Sortare topologică (ordinea descrescătoare a nodurilor după timpul de finish)

1, 4, 5, 6, 2, 3, 7

2) Kahn

node	1	2	3	4	5	6	7
in_degree	0	1	3	1	1	1	3

Queue

~~1~~
~~2~~
~~4~~
~~5~~
~~6~~
~~3~~

1) se scoate 1 din queue => vecini: 2, 4
(se scade in-degree pt. ei)

node	1	2	3	4	5	6	7
in_degree	0	0	3	0	1	1	3

in_degree[2] = in_degree[4] == 0
=> se adaugă 2, 4 la queue

2) se scoate 2 => vecini: 3, 7

node	1	2	3	4	5	6	7
in_degree	0	0	2	0	1	1	2

Ambele in_degree > 0 => nu se adaugă la queue

3) se scoate 4 => vecini: 3, 5

node	1	2	3	4	5	6	7
in_degree	0	0	1	0	0	1	2

in_degree[5] == 0 => se adaugă 5 la coadă

4) se scoate 5 => vecini: 6

node	1	2	3	4	5	6	7
in_degree	0	0	1	0	0	0	2

in_degree[6] == 0 => se adaugă 6

5) se scoate 6 => vecini: 3, 7

(10)

nod	1	2	3	4	5	6	7
in-degree	0	0	0	0	0	0	1

in-degree[3] == 0 => se adaugă 3

6) se scoate 3 => vecini: 7

nod	1	2	3	4	5	6	7
in-degree	0	0	0	0	0	0	0

Ordinea în care au fost scoase = sortarea topologică

1, 2, 4, 5, 6, 3, 7

OBSERVAȚII

1) Sortarea topologică nu este unică.

2) Care este numărul minim / maxim de sortări topologice?

↓ depinde de graf (dacă este ciclic, poate fi 0) și

↳ greu de calcul în timp polinomial (se poate adapta pentru a obține toate sorturile topologice - backtracking se consideră pe rând fiecare nod curent cu in-degree == 0)

COMPONENTE CONEXE / TARI CONEXE

Pt. grafuri neorientate avem COMPONENTE CONEXE = submulțimi maximale de noduri cu proprietatea că $(\forall) x, y \in V$, $(\exists)$ u ⇝ v (drum) (CC)

(*) Un graf este conex dacă are o singură componentă conexă

COMPONENTĂ TARE CONEXĂ - grafuri orientate (CTC)

$(\forall)$ 2 noduri u, v $\begin{cases} (\exists)\ u \rightsquigarrow v \\ (\exists)\ v \rightsquigarrow u \end{cases}$ (u reachable din v și v reachable din u)

1) DETERMINAREA CC – DFS și de fiecare dată când găsim un nod fără părinte, avem o nouă CC

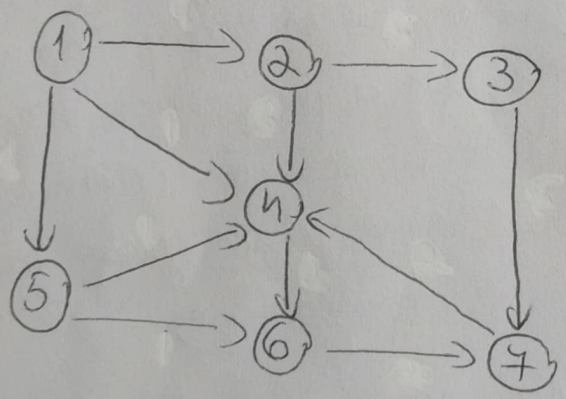
2) -11- CTC

(12)

2.1) KOSARAJU → COMPLEXITATE $O(|V|+|E|)$

1) DFS(G) pentru a afla timpii de finish pentru fiecare nod
2) Calculăm G^T (graful transpus)
   Dacă $G=(V,E)$ atunci $G^T=(V,E^T)$
   pt. $\forall (u,v) \in E$, în E^T punem (v,u)
3) DFS(G^T) în care considerăm nodurile după ordinea descrescătoare a timpilor de finish. (pasul 1 = sortare topologică a nodurilor)

S1.2 2017 CB) de evidențiat CTC prin aplicarea Kosaraju

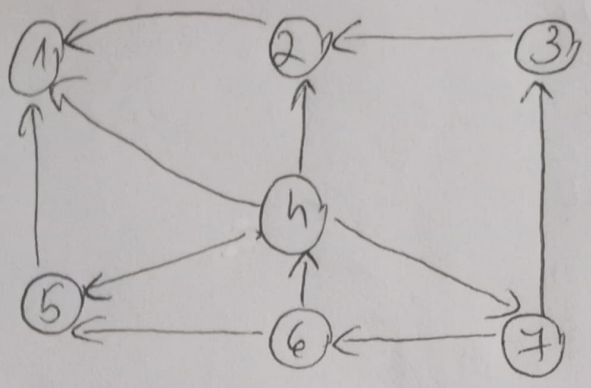


node	adj[node]
1	2,4,5
2	3,4
3	7
4	6
5	4,6
6	7
7	4

1) DFS(G): DFS(1): 1 → 14
DFS(2): 2 → 11
DFS(3): 3 → 10
DFS(7): 4 → 9
DFS(4): 5 → 8
DFS(6): 6 → 7
DFS(5): 12 → 13

node	1	2	3	4	5	6	7
f[node]	14	11	10	8	13	7	9

2) G^T:



node	adj[node]
1	-
2	1
3	2
4	1, 2, 5, 7
5	1
6	4, 5
7	3, 6

3) DFS(G^T) după ordinea descrescătoare a timpilor de la 1: 1, 5, 2, 3, 7, 4, 6 (finished)

DFS(1): 1 → 2
DFS(5): 3 → 4
DFS(2): 5 → 6
DFS(3): 7 → 8
DFS(7): 9 → 14
DFS(6): 10 → 13
DFS(4): 11 → 12

CTC: {1}, {5}, {2}, {3}, {4, 6, 7}

(13)

2.2) TARJAN

→ folosește o singură parcurgere DFS și pune nodurile pe o stivă
→ fiecare CTC are o rădăcină (pentru care low_link[node] == found[node]), când ajungem la rădăcina unei CTC, toate elementele din CTC sunt deja pe stivă

found[node] = timpul descoperirii nodului
low_link[node] = min { found[x] | x == node sau
x descendent al lui node sau
node descendent al lui x }

```
TARJAN ( G=(nodes, adj))
    FOR node ∈ nodes
        p[node] = NULL
        found[node] = -1
        low_link[node] = ∞
    CTCs = [ ]
    current_CTC = new Stack()
    timestamp = 0
    FOR node ∈ nodes
        if p[node] == NULL
            p[node] = node // convenție (părintele root-ului = root-ul)
            DFS(node, G, timestamp, current_CTC, CTCs, found,
                low_link, p)
```

COMPLEXITATE: $O(|E|+|V|)$

(14)

```
DFS( node, G, timestamp, current_CTC, CTCs, found, low_link, p)
    timestamp++
    found[node] = timestamp
    low_link[node] = found[node]
    current_CTC.push(node)
    FOR neigh ∈ adj[node]
        if p[neigh] != NULL // este deja descoperit
            if neigh ∈ current_CTC // propagă low_link numai pt. noduri din CTC
                low_link[node] = min(low_link[node], current
                                     low_link[neigh])
                continue
```

```
        p[neigh] = node
        DFS(neigh, G, timestamp, current_ctc, CTCs, found, low_link, p)
    low_link[node] = min(low_link[node], low_link[neigh])

// după parcurgerea tuturor vecinilor
// node este root-ul CTC-ului dacă found[node] == low_link[node]
if found[node] == low_link[node]
    CTCs.push(current_ctc.copy())
    current_ctc = []
```

(15)

ALTE APLICAȚII TARJAN

1) PUNCTE DE ARTICULAȚIE → eliminarea lui duce la mărirea numărului de componente conexe (GRAF NEORIENTAT)

$u \in V$ punct de articulație dacă $(\exists)\ x, y \in V$, $u \neq x$, $u \neq y$, $x \neq y$

a.î. $(\forall)$ cale de la x la y $x \rightsquigarrow y$ TRECE PRIN u

Adaptăm Tarjan căutând toate nodurile care satisfac una dintre următoarele condiții:

1) node == root-ul unei CC și are >1 copii
2) node ≠ root-ul unei CC și $(\exists)$ neigh inițial: nedescoperit
   dar pentru care low_link[neigh] ≥ found[node] după parcurgere
   (cel mai mic timestamp al nodurile accesibile din neigh ≥ timestamp-ul lui node

2) PUNȚI / MUCHII CRITICE → eliminarea muchiei duce la creșterea numărului de componente conexe (tot graf neorientat)

(u, v) punte dacă $(\exists)\ x, y \in V,\ x \neq u,\ x \neq v,\ y \neq u,\ y \neq v$

a.î. (↯) drum de la x la y trece prin (u, v)

Căutăm cu Tarjan toate perechile (node, neigh) cu

low_link[neigh] > found[node]

(16)

3) COMPONENTE BICONEXE

→ un graf neorientat este biconex dacă are o singură componentă biconexă

→ COMPONENTĂ BICONEXĂ = (↯) nod aș elimina din submulțime, nodurile rămase sunt încă conectate

(submulțime maximală de noduri fără puncte de articulație)

Se folosește Tarjan și de fiecare dată când se găsește un punct de articulație, toate muchiile de până acum (de forma (node, neigh)) formează o componentă biconexă.

(dacă aș elimina nodul, tot subarborele dat de node-neigh-... rămâne deconectat)

DRUMURI MINIME

- fiecare muchie are asociat un cost $w[u][v]$
- costul unei căi $p = (v_1, \dots, v_k)$ este $w(p) = \sum_{i=1}^{k-1} w[v_i][v_{i+1}]$

VARIANTE

V1) (sursă, destinație) - SINGLE PAIR (folosind V2)
V2) (sursă, (∀) destinație) - SINGLE SOURCE (Dijkstra, Bellman-Ford)
V3) ((∀) sursă, destinație) - SINGLE DESTINATION (facem G^T și după V2)
V4) ((∀) sursă, (∀) destinație) - ALL PAIRS (Floyd-Warshall, Johnson)

PROPRIETĂȚI UTILE

cea mai mică distanță de la s la u

1) $\delta[s][v] = \delta[s][u] + w[u][v]$ pentru drumul $s \overset{\text{OPTIM}}{\rightsquigarrow} u \to v$
2) $\delta[s][v] \leq \delta[s][u] + w[u][v], (\forall)(u,v) \in E$

RELAXAREA ARCELOR

→ căutăm noduri intermediare u prin care să reducem costul drumului de la sursă la v: $d[source][v] > d[source][u] + w[u][v]$

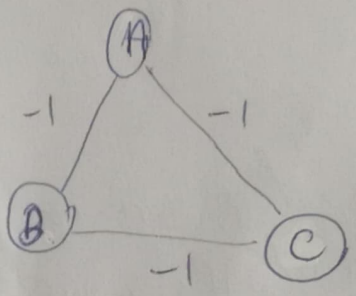
1) DIJKSTRA - se folosește numai pentru COSTURI POZITIVE

```
DIJKSTRA (source, G = (nodes, adj))
    FOR node ∈ nodes
        d[node] = ∞
        p[node] = NULL

    pq = new PriorityQueue()
    d[source] = 0
    pq.push((source, d[source]))
```

```
WHILE !pq.isEmpty()
    node = pq.pop() // cu d[node] minim
    FOR neigh ∈ adj[node]
        IF d[node] + w[node][neigh] < d[neigh]
            d[neigh] = d[node] + w[node][neigh]
            p[neigh] = node
            pq.push((neigh, d[neigh]))
return d, p
```


COSTURI NEGATIVE



source = A

queue

(A, 0)

d:

0	∞	∞

1) scoatem nod = (A, 0), vecini: B, C

Obtinem un cost mai bun (-1) și pt. B și pt. C

$d[A] + w[A][B] = 0 + (-1) < \infty = d[B]$

$d[A] + w[A][C] = 0 + (-1) < \infty = d[C]$

⇒ actualizăm costurile și punem nodurile în coadă

queue:

(B, -1)	(C, -1)

d:

0	-1	-1

2) se scoate $(B, -1)$ cu vecinii A, C

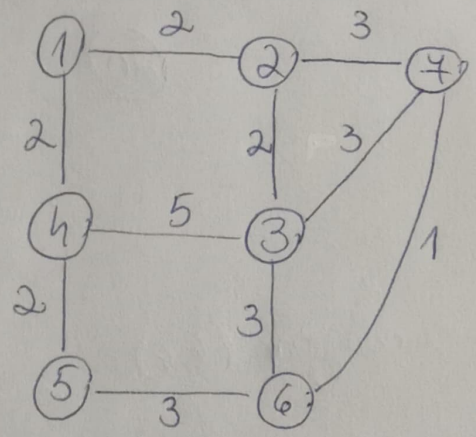
$d[B] + w[B][A] = (-1) + (-1) = -2 < 0 = d[A]$

$d[C] + w[C][B] = (-1) + (-1) = -2 < -1 = d[C]$

$\Rightarrow$ se pun în coadă $(A, -2), (C, -2)$

Algoritmul o să ciclească pentru că o să adauge constant noi noduri în coadă pentru un cost $\to -\infty$.

S1.3 2017 CC)



queue

(1,0)

d:

0	∞	∞	∞	∞	∞	∞
1	2	3	4	5	6	7

2 2

Dijkstra din nodul 1

1) scoatem (1,0)

$d[1] + w[1][2] = 0 + 2 < \infty = d[2] \Rightarrow$ queue.push((2,2))

$d[1] + w[1][4] = 0 + 2 < \infty = d[4] \Rightarrow$ — " — (4,2)

queue: (2,2) | (4,2)

d:

0	2	∞	2	∞	∞	∞
		4				5

2) scoatem (2,2)

$d[2] + w[2][7] = 2 + 3 < \infty = d[7] \Rightarrow$ adăugăm (7,5)

$d[2] + w[2][3] = 2 + 2 < \infty = d[3]$ — " — (3,4)

queue: (4,2) | (7,5) | (3,4)

4

d:

0	2	4	2	∞	∞	5

3) scoatem (4,2)

$d[4] + w[4][5] = 2 + 2 < \infty = d[5] \Rightarrow$ adăugăm (5,4)

queue: ~~(7,5)~~ ~~(3,4)~~ (5,4)

d: | 0 | 2 | 4 | 2 | 4 | ~~∞~~ 7 | 5 |

4) scoatem (3,4)

$d[3] + w[3][6] = 4 + 3 < \infty = d[6] \Rightarrow$ adăugăm (6,7)

queue: ~~(7,5)~~ ~~(5,4)~~ (6,7)

d: | 0 | 2 | 4 | 2 | 4 | ~~7~~ 6 | 5 |

(20)

5) scoatem (5,4)

6) scoatem (7,5)

$d[7] + w[7][6] = 5 + 1 < 7 = d[6] \Rightarrow$ adăugăm (6,6)

queue ~~(6,7)~~ ~~(6,6)~~

d | 0 | 2 | 4 | 2 | 4 | 6 | 5 |

7) scoatem (6,6)

COMPLEXITATE DIJKSTRA

→ folosim metodele { isEmpty, push, pop } care depind de implementarea cozii

→ heap liniar - complexitate temporală: $O(|E| \log |V|)$ (log |V| ↔ inserare nodului în coadă)

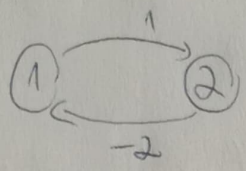
↑ încercă relaxarea fiecărei muchii

→ cu heap Fibonacci $O(|V| \log |V| + |E|)$

→ complexitate spațială $O(n)$

2) BELLMAN-FORD - muchiile pot să aibă cost negativ, dar nu trebuie să existe cicluri de cost negativ

Dacă există cicluri de cost negativ atunci este posibil să nu obțin drumul minim la final



$d[1][2] \to -\infty$

Orice drum minim are max $|V|-1$ muchii (nu poate conține ciclu). Dacă ar fi avut un ciclu atunci putem să îl eliminăm și reducem costul $\Rightarrow$ drumul inițial nu mai este minim

Algoritmul face $|V|-1$ relaxări. Dacă la final încă se mai pot relaxa arcele $\Rightarrow$ ($\exists$) cicluri de cost negativ

```
BELLMAN-FORD (source, G=(nodes, edges))
    FOR node ∈ nodes
        d[node] = ∞
        p[node] = NULL
    FOR i=1:|nodes|-1
        FOR (node, neigh) ∈ edges
            IF d[node] + w[node][neigh] < d[neigh]
                d[neigh] = d[node] + w[node][neigh]
                p[neigh] = node
    FOR (node, neigh) ∈ edges
        IF d[node] + w[node][neigh] < d[neigh]
            // CICLU NEGATIV
            return null
```

COMPLEXITATE temporală $O(|V| \cdot |E|)$
spațială $O(1)$

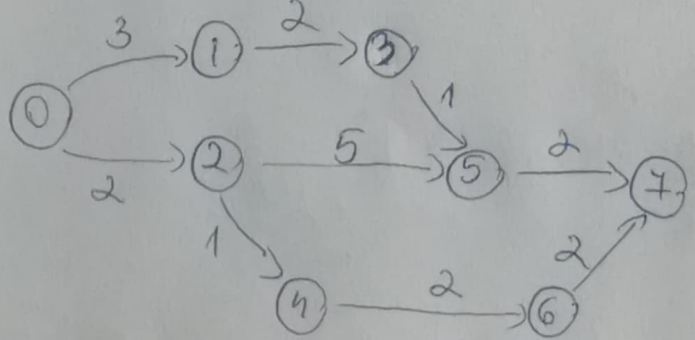
CAZ PARTICULAR: DAG

- pentru un DAG (directed acyclic graph) dacă am un path de la u la v atunci u îl precede pe v în sortarea topologică
- parcurgem nodurile în ordinea dată de sortarea topologică și e suficient să relaxăm fiecare muchie o singură dată

```
FOR node ∈ nodes
    d[node] = ∞
    p[node] = NULL
d[source] = 0
sortedVertices = TOPSORT(G)
FOR node ∈ sortedVertices
    FOR neigh ∈ adj[node]
        IF d[node] + w[node][neigh] < d[neigh]
            d[neigh] = d[node] + w[node][neigh]
            p[neigh] = node
```


Complexitatea temporală $O(|V|+|E|)$
spațială $O(|V|)$

S1.3 2019 CB)



sursa 0
destinație 7

d:

0	~~∞~~ 3	~~∞~~ 2	~~∞~~ 5	~~∞~~ 3	~~∞~~ 6	~~∞~~ 5	~~∞~~ 7

iterația 1:

(0,1): $d[0] + w[0][1] = 0 + 3 < \infty = d[1] \Rightarrow d[1] = 3$

(0,2): $d[0] + w[0][2] = 0 + 2 < \infty = d[2] \Rightarrow d[2] = 2$

(1,3): $d[1] + w[1][3] = 3 + 2 < \infty = d[3] \Rightarrow d[3] = 5$

(2,5): $d[2] + w[2][5] = 2 + 5 < \infty = d[5] \Rightarrow d[5] = 7$

(2,4): $d[2] + w[2][4] = 2 + 1 < \infty = d[4] \Rightarrow d[4] = 3$

(4,6): $d[4] + w[4][6] = 3 + 2 < \infty = d[6] \Rightarrow d[6] = 5$

(6,7): $d[6] + w[6][7] = 5 + 2 < \infty = d[7] \Rightarrow d[7] = 7$

(3,5): $d[3] + w[3][5] = 5 + 1 < 7 = d[5] \Rightarrow d[5] = 6$

(5,7): $d[5] + w[5][7] = 6 + 2 > 7 = d[7]$

d:

0	3	2	5	3	6	5	7

iterația 2:

(0,1): – (4,6): –

(0,2): – (6,7): –

(1,3): – (3,5): –

(2,5): – (5,7): –

(2,4): –

Se fac $|V| - 1 = 4$ iterații, dar de la a 2-a nu se mai schimbă valorile lui d.

3) ROY - FLOYD / FLOYD - WARSHALL folosește o dinamică pentru a calcula distanța minimă între orice pereche de noduri

INIȚIALIZARE: $d[i][j]$ = distanța minimă de la i la j

$$\begin{cases} 0, & i = j \\ w[i][j], & \text{dacă } (i,j) \in E \\ \infty, & \text{altfel} \end{cases}$$

RECURENȚĂ: $d[i][j] = \min(d[i][j], d[i][k] + d[k][j]), i \neq j$

$i \to \dots \to k \to \dots \to j$ se încearcă relaxarea prin noduri intermediare k

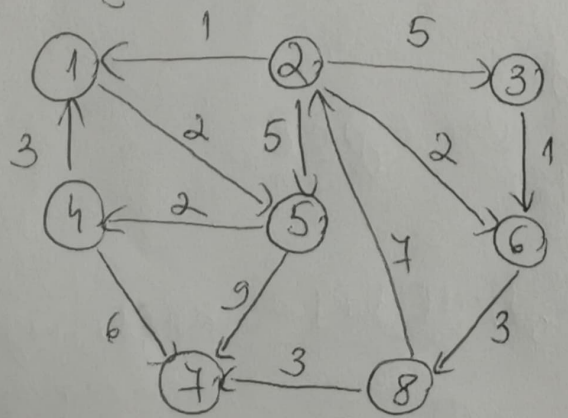
p[i][j] = parintele lui j pe calea de la i la j

```
ROY-FLOYD (G=(nodes, adj))
    FOR i ∈ nodes
        FOR j ∈ nodes
            d[i][j] = { 0, i=j
                      { w[i][j], j ∈ adj[i]
                      { ∞, altfel
            p[i][j] = { i, j ∈ adj[i]
                      { NULL, altfel
    FOR k ∈ nodes
        FOR i ∈ nodes
            FOR j ∈ nodes
                IF d[i][k] + d[k][j] < d[i][j]
                    d[i][j] = d[i][k] + d[k][j]
                    p[i][j] = p[k][j]
```


COMPLEXITATE temporală $O(|V|^3)$

spațială $O(1)$

S1.2 rezolvați)



D(0), D(1), D(2)

$D(0) =$

0	∞	∞	∞	2	∞	∞	∞
1	0	5	∞	5	2	∞	∞
∞	∞	0	∞	∞	1	∞	∞
3	∞	∞	0	∞	∞	6	∞
∞	∞	∞	2	0	∞	9	∞
∞	∞	∞	∞	∞	0	∞	3
∞	∞	∞	∞	∞	∞	0	∞
∞	7	∞	∞	∞	∞	3	0

$d[i][j] = \min(d[i][j], d[i][k] + d[k][j]), (\forall) i,j$

$K = 1$

$D(1) =$

0	∞	∞	∞	2	∞	∞	∞
1	0	5	∞	3	2	∞	∞
∞	∞	0	∞	∞	1	∞	∞
3	∞	∞	0	5	∞	6	∞
∞	∞	∞	2	0	∞	9	∞
∞	∞	∞	∞	∞	0	∞	3
∞	∞	∞	∞	∞	∞	0	∞
∞	7	∞	∞	∞	∞	3	0

$d[i][j] = \min(d[i][j], d[i][1] + d[1][j])$

$d[i][1]$ – primo coloană; $d[1][j]$ – primă linie

pt. $(\forall) i,j$

$K = 2$

$D(2) =$

0	∞	∞	∞	2	∞	∞	∞
1	0	5	∞	3	2	∞	∞
∞	∞	0	∞	∞	1	∞	∞
3	∞	∞	0	5	∞	6	∞
∞	∞	∞	2	0	∞	9	∞
∞	∞	∞	∞	∞	0	∞	3
∞	∞	∞	∞	∞	∞	0	∞
8	7	12	∞	10	9	3	0

$d[i][j] = \min(d[i][j], d[i][2] + d[2][j])$

pt. $(\forall) i,j$

(*) INCHIDERE TRANZITIVĂ

Fie un graf $G = (V, E)$ vrem să aflăm $E^*(i,j) = \begin{cases} 1, \text{ dacă } (\exists) \text{ cale de la } i \text{ la } j \ (i \rightsquigarrow j) \\ 0, \text{ altfel} \end{cases}$

$G = (V, E^*)$ – numește închiderea tranzitivă

Putem modifica Floyd-Warshall simplu:

Când avem cale de la i la j?

→ când am găsit-o deja (ex: există muchie între i și j)

→ avem un nod intermediar k a.î. (∃) cale de la i la k și (∃) cale de la k la j

$$E^*(i,j) = \underbrace{E^*(i,j)}_{(1)} \vee \underbrace{(E^*(i,k) \wedge E^*(k,j))}_{\text{trebuie să am obligatoriu amândouă părți}}$$

(i) → .. → (k) → .. → (j)

Inițializare: $E^*(i,j) = \underbrace{(i==j)}_{(*)\text{ nod e conectat cu el însuși}} \vee ((i,j) \in E)$

(26)

4) JOHNSON

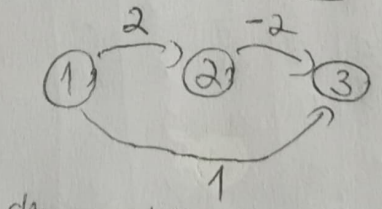
→ aplicăm Dijkstra din fiecare sursă (costurile trebuie să fie obligatoriu pozitive)

→ IDEE: transformarea grafului a.î. toate costurile să fie pozitive

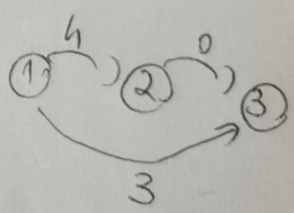
→ Cum facem costurile pozitive?

1) Adunăm cel mai mic cost (în modul) — NU este okay

exemplu pe care pică:



devine (+2)



drum minim de la 1 la 3 este 1 → 2 → 3

drumul minim este 1 → 3

nu este okay pt. că drumurile nu au toate aceeași lungime (pe unde adaug mai mult decât pe altele ⇒ strică structura grafului)

2) Funcție pe noduri: pt. (∀) $u \in V$, calculez $h(u)$

și costul este $\hat{w}[u][v] = w[u][v] + h(u) - h(v)$

$\hat{w}[u][v] = w[u][v] + h(u) + h(v) \geq 0, \ (\forall) u, v \in V$

1) Ce nu schimbă <u>drumurile</u> (27)

Dacă inițial aveam un path $p = (u_1, \dots, u_k)$ cu un cost $w[p] = \sum_{i=1}^{k-1} w[u_i][u_{i+1}]$ atunci noul cost este

$$\hat{w}[p] = \sum_{i=1}^{k-1} \hat{w}[u_i][u_{i+1}] = \sum_{i=1}^{k-1} \left(w[u_i][u_{i+1}] + h(u_i) - h(u_{i+1}) \right)$$

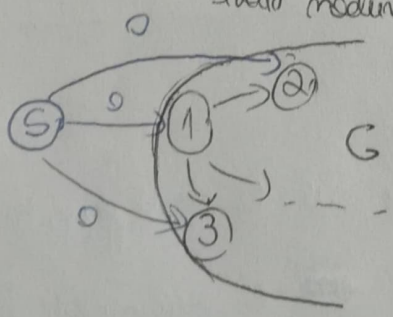
$$= w[p] + h(u_1) - \cancel{h(u_2)} + \cancel{h(u_2)} - \cancel{h(u_3)} + \dots + \cancel{h(u_{k-1})} - h(u_k)$$

$$= w[p] + \underbrace{h(u_1) - h(u_k)}_{\text{nu depinde de nodurile intermediare}}$$

2) Cum calculăm h a.î. $\hat{w}[u][v] \geq 0$

$h[u] = \delta[s][u]$ → poate fi negativă dacă ($\exists$) muchii de cost negativ (ex: s → 1 cu 0, 1 → 2 cu -2, s → 2 cu 0)

↳ adăugăm un nou nod s pe care îl conectăm la toate nodurile prin muchii de cost 0



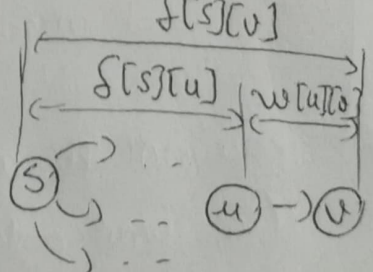
Verificăm condiția $\hat{w}[u][v] \geq 0, \ (\forall) u, v$

$w[u][v] + h(u) - h(v) \geq 0$

$w[u][v] + \delta[s][u] - \delta[s][v] \geq 0$

$\delta[s][v] \leq \delta[s][u] + w[u][v]$ (A)

(egal atunci când drumul minim trece prin u)



Folosim Bellman-Ford pt. a calcula $\delta[s][u]$ (muchiile pot să aibă cost negativ)

```
JOHNSON (G=(V,E))
    G' = ... // adăugăm s și muchii de cost 0 de la s la toate celelalte noduri
    h = BELLMAN-FORD(G')
    FOR (u,v) ∈ E
        w[u][v] += h[u] - h[v]
    FOR source ∈ V
        d_current, p_current = DIJKSTRA(source, G)
        FOR u ∈ V
            d[source][u] = d_current[u] + h[u] - h[source]
            p[source][u] = p_current[u]
```

(28)

trebuie să scădem h[source] - h[u] pt. a ajunge la costul inițial al căii

COMPLEXITATE temporală $O(\overbrace{|E|\cdot|V|}^{\text{Bellman-Ford}} + \overbrace{|E|\cdot|V|\cdot\log|V|}^{\text{Dijkstra}}) = O(|E|\cdot|V|\log|V|)$

spațială $O(|E|+|V|)$

ARBORI MINIMI DE ACOPERIRE

Pt. un graf neorientat $G=(V,E)$ cu $w: E \to W$ funcție de cost.

Un arbore minim de acoperire este o submulțime

- $T \subseteq E$ care conectează toate nodurile
  + minimizează costul $\sum_{(u,v)\in T} w[u][v]$

→ sau PĂDURE DE ARBORI MINIMI DE ACOPERIRE (câte unul pentru fiecare componentă conexă)

ALGORITM GENERIC

$T = \emptyset$

WHILE T nu formează un MST

alog o muchie (u,v) SIGURĂ

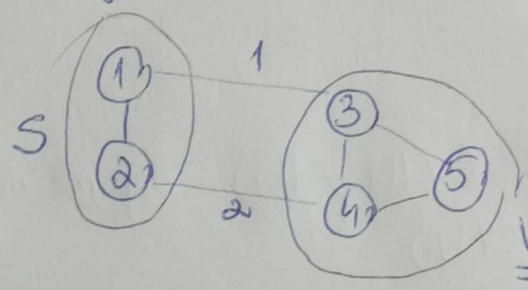
$T = T \cup \{(u,v)\}$

↳ câte muchii are un MST? $|V|-1$ (orice arbore care trece prin n noduri are n-1 muchii)

↳ cum alegem o muchie sigură?

Pentru setul T căutăm o partiționare $(S, V \setminus S)$ a.î. nicio muchie din T nu taie frontiera dintre S și $V \setminus S$

exemplu:



$V \setminus S$

↳ partiție peste noduri, nu muchii

Muchia sigură = muchia de cost minim care taie frontiera

↳ cum alog muchia / partiționarea depinde de algoritm

1) KRUSKAL

↳ fiecare nod formează o mulțime inițial

↳ parcurge nodurile în ordinea crescătoare a costului

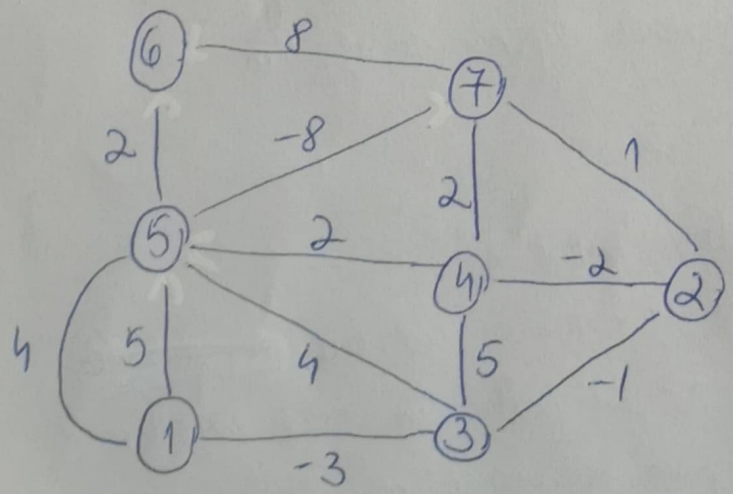
- dacă capetele muchiei nu fac parte din aceeași mulțime
  - ⇒ reunește mulțimile prin adăugarea muchiei

```
KRUSKAL(G=(V,E))
    T = ∅
    FOR v ∈ V
        MAKE-SET(v)
    sort(E)
    FOR (u,v) ∈ E                    ← dacă sunt deja în aceeași componentă nu are sens să adaug muchia că fac ciclu
        IF FIND-SET(u) ≠ FIND-SET(v)
            T = T ∪ {(u,v)}
            UNION(u,v)
```

(30)

COMPLEXITATE timpurie $O(|E| \log |E|)$ - depinde de folosită (Disjoint Set)
spațială $O(|V|)$

Sl. 4 2014 CA)



aplicat Kruskal

++ mentionat muchiile alese

① ② ③ ④ ⑤ ⑥ ⑦

~~Ordine muchii: (5,7), (1,3), (4,2), (3,2), (7,2), (5,6), (4,7), (3,5), (,), (1,5)~~

Ordine muchii (+ cost):
(5,7) - cost -8
(1,3) -11- -3
(2,4) -11- -2
(2,3) -11- -1
(4,5), (4,7), (5,6) -11- -2 ← (2,7) -11- 1
(1,5), (3,5) -11- 4
(1,5), (3,4) -11- 5
(6,7) -11- 8

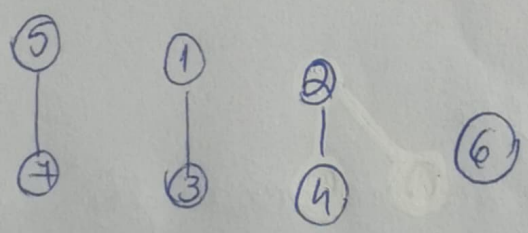
$T = \emptyset$

(31)

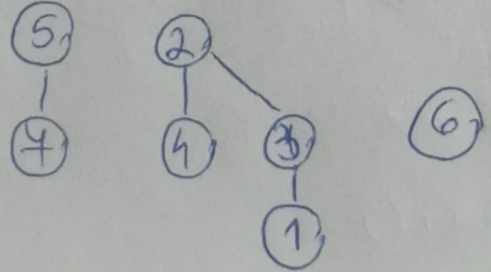
1) (5,7) => UNION (5,7), $T = \{(5,7)\}$
2) (1,3) => UNION (1,3), $T = \{(5,7), (1,3)\}$
3) (2,4) => UNION (2,4), $T = \{(5,7), (1,3), (2,4)\}$
4) (2,3) => UNION (2,3), $T = \{(5,7), (1,3), (2,4), (2,3)\}$
5) (4,5) => UNION (4,5), $T = \{(5,7), (1,3), (2,4), (2,3), (4,5)\}$
6) (4,7)
7) (5,6) => UNION (5,6), $T = \{(5,7), (1,3), (2,4), (2,3), (4,5), (5,6)\}$
8) (1,5)

---- (T are deja 6 muchii deci nu se mai modifică)

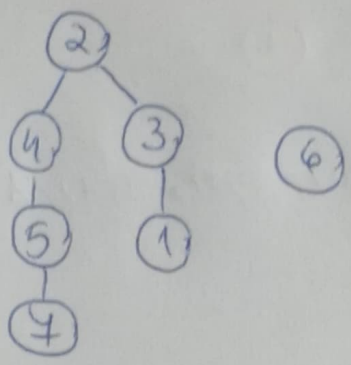
după 1+2+3)



4)

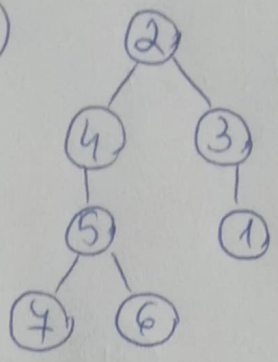


5)



6

7)



32

2) PRIM ~ asemănător Dijkstra

→ aleg random o sursă și încerc să extind prin conectarea arborelui parțial de un nod izolat

→ toate nodurile rămase sunt puse într-un priority queue și sunt sortate în funcție de costul minim al oricărui muchii care leagă nodul de orice alt nod din arbore (vreau să conectez noul nod la unul existent, dar printr-un cost minim)

```
PRIM (G=(V,E))
    FOR v ∈ V
        used[v] = false
        d[v] = ∞ //distanța minimă față de oricare nod din
        p[v] = NULL // arborele parțial
    MST = [] // muchiile alese
    PQ = new PriorityQueue()
```

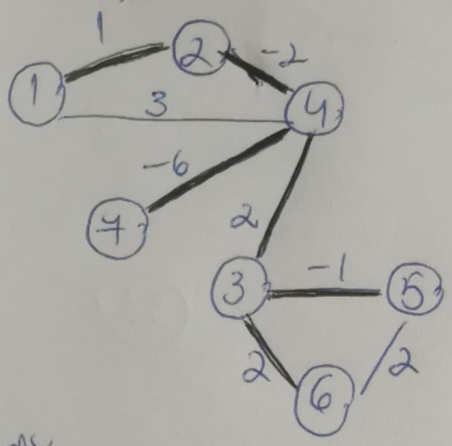
```
cost = 0
d[source] = 0 ; used[source] = true
pq: push((source, d[source]))

WHILE !pq.isEmpty()
    node = pq.pop()
    IF used[node]
        continue
    //adaugam muchia prin p[node] (muchia de cost minim
    IF p[node] != NULL                                  este (p[node], node))
        cost += w[p[node]][node] // costul MST-ului
        MST.push((p[node], node))
    FOR neigh ∈ adj[node]
        IF !used[neigh] && w[node][neigh] < d[neigh]
            //relaxare tuturor vecinilor
            d[neigh] = w[node][neigh]
            p[neigh] = node
            pq.push((neigh, d[neigh]))
```


COMPLEXITATE temporală $O(|E| \log |V|)$

spațială $O(|V|)$

S1.3 2020)



nâdă cuva = 2

source = 2

d: | ∞ | 0 | ∞ | ∞ | ∞ | ∞ | ∞ |

pq: | (2,0) | _ | _ |

→ (node, d[node])

1) scot (2,0)

d: | 1 | 0 | ∞ | -2 | ∞ | ∞ | ∞ |

p: | 2 | | | 2 | | | |

pq: | (1,2) | (4,-2) |

(p[node], node)

2) scot (4,-2) => adaug (2,4) la MST

d: | 1 | 0 | 2 | -2 | ∞ | ∞ | -6 |

p: | 2 | | 4 | 2 | | | 4 |

pq: | (1,2) | (3,2) | (7,-6) |

3) scot (7,-6) => adaug (4,7) la MST

d: | 1 | 0 | 2 | -2 | ∞ | ∞ | -6 |

p: | 2 | | 4 | 2 | | | 4 |

pq: | (1,2) | (3,2) |

4) scot (1,2) => adaug (2,1) la MST

d: | 1 | 0 | 2 | -2 | | | -6 |

p: | 2 | | 4 | 2 | | | 4 |

pq: | (3,2) | | |

5) scot (3,2) => adaug (4,3) la MST

d: | 1 | 0 | 2 | -2 | -1 | 2 | -6 |

p: | 2 | | 4 | 2 | 3 | 3 | 4 |

pq: | (5,-1) | (6,2) |

3.4

6) scot (5,-1) => adaug (3,5) la MST

d: | 1 | 0 | 2 | -2 | -1 | 2 | -6 |

p: | 2 | | 4 | 2 | 3 | 3 | 4 |

pq: | (6,2) |

7) scot (6,2) => adaug (3,6) la MST

d: | 1 | 0 | 2 | -2 | -1 | 2 | -6 |

p: | 2 | | 4 | 2 | 3 | 3 | 4 |

pq: |

SECOND-BEST MST

(al doilea cel mai bun arbore minim de acoperire din punct de vedere al costului)

→ este suficient să modificăm una dintre muchiile MST-ului

1) Aflăm MST-ul.

2) Luăm pe rând toate muchiile care nu sunt în MST și le adăugăm la MST. Se va forma un ciclu și căutăm muchia de cost maxim diferită de cea adăugată (dacă am păstra noua muchie, care este cel mai mic cost al arborelui pe care îl putem obține?)

3) Pentru fiecare muchie care nu face parte din MST calculăm

$$w[e_1] - \max\{ w[e_2] \mid e_2 \in MST \cup \{e_1\} \}$$

încercăm să adăugăm muchia e_1 — și se scade greutatea maximă — se adaugă muchia la MST

4) Se ia muchia care minimizează valoarea de mai sus.

5) Muchia găsită se scoate din MST + se adaugă muchia de cost maxim de la pasul 3

FLUX

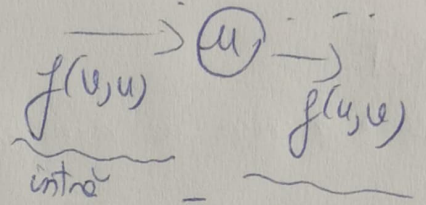
$G=(V,E)$ graf orientat cu 2 noduri speciale S = sursă / producător
T = terminal / consumator

$c[u][v] \geq 0$, $(\forall)(u,v) \in E$ – FĂRĂ SELF LOOPS
↳ capacitate
$(u,u) \notin E$, $(\forall) u \in V$

Fluxul este o funcție $f: V \times V \to \mathbb{R}$ cu proprietățile:

1) CAPACITY CONSTRAINT: $0 \leq f(u,v) \leq c(u,v)$, $(\forall) u,v \in V$

2) FLOW CONSERVATION: $\sum_{v \in V} f(v,u) = \sum_{v \in V} f(u,v)$, $(\forall) u \in V \setminus \{s,t\}$



ce intră = ce iese

3) ANTISIMETRIE: $f(u,v) = -f(v,u)$, $(\forall) u,v \in V$

2) poate fi rescrisă ca $\sum_{v \in V} f(u,v) = -\sum_{v \in V} f(v,u)$

$\sum_{v \in V} f(u,v) = 0$

Valoarea unui flow f: $|f| = \sum_{v \in V} f(s,v) - \sum_{v \in V} f(v,s)$

Capacitate reziduală: $c_R(u,v) = \underbrace{c(u,v)}_{\text{maxim cât pot duce}} - \underbrace{f(u,v)}_{\text{cât duc acum}}$

Rețea reziduală: păstrez numai (u,v) pt. care $c_R(u,v) > 0$ (îl mai pot exploata)

$G_R = (V, E_R)$, $E_R = \{(u,v) \in E \mid c_R(u,v) > 0\}$

↓ câte muchii are?

1) toate muchiile cu $c(u,v) = f(u,v)$ sunt scoase
2) dacă $f(u,v) < c(u,v)$ atunci $c_R(u,v) > 0$ și e inclusă
3) dacă $(u,v) \notin E$ atunci $c(u,v)$

$c_R(u,v) = -f(u,v) = f(v,u)$. Dacă $(v,u) \in E \Rightarrow (u,u) \in E_R$

$$c_R(u,v) = \begin{cases} c(u,v) - f(u,v), & (u,v) \in E \\ f(v,u), & (u,v) \notin E \text{ și } (v,u) \in E \\ 0, & \text{altfel} \end{cases}$$

Worst case adaug de 2 ori mai multe muchii ($|E_R| \leq 2 \cdot |E|$)

CAPACITATEA REZIDUALĂ a unui drum de ameliorare p este

$$c_R(p) = \min \{ c_R(u,v) \mid (u,v) \in p \}$$

```
FORD-FULKERSON (G,s,t)
    FOR (u,v) ∈ E
        f(u,v) = 0
    WHILE (G_R mai are căi de ameliorare)
        // caut calea p
        c_R(p) = min { c_R(u,v) / (u,v) ∈ p }
        FOR (u,v) ∈ p
            IF (u,v) ∈ E
                f(u,v) += c_R(p)
            ELSE
                f(v,u) -= c_R(p)
```

(37)

Cum aleg calea de ameliorare? prin BFS la Edmonds-Karp
↓pt. a alege calea cea mai scurtă

EDMONDS-KARP → BFS cât timp $f(u,v) < c(u,v)$

```
BFS(G,s,t,f,c)
    FOR u ∈ V
        p[u] = NULL
```

```
queue = new Queue()
queue.push(source)
p[source] = NULL

WHILE !queue.isEmpty()
    node = queue.pop()
    FOR neigh ∈ adj[node]
        IF p[neigh] != NULL && f[node][neigh] < c[node][neigh]
            p[neigh] = node
            queue.push(neigh)

IF p[t] == NULL
    // nu mai există căi de ameliorare

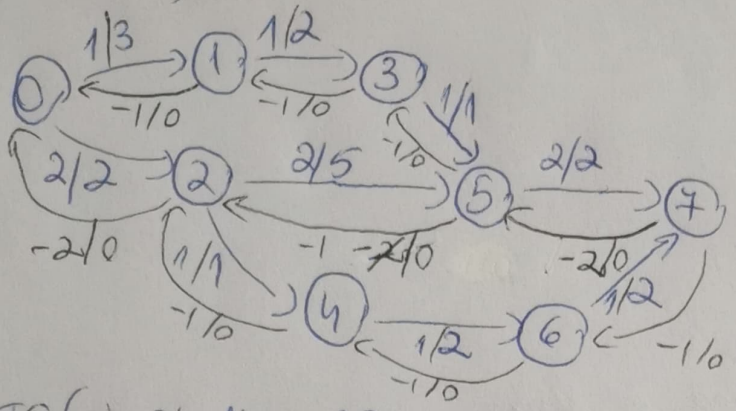
return path // rebuild
```



```
EDMONDS-KARP(G, s, t)
    totalFlow = 0
    f(u,v) = 0, (∀) u,v ∈ V
    WHILE (p = BFS(G, s, t, f, c))
        IF !p
            return totalFlow
        c_R(p) = min{ c[u][v] - f[u][v] / (u,v) ∈ p }
        FOR (u,v) ∈ p
            f[u][v] += c_R(p)
            f[v][u] -= c_R(p)
```

COMPLEXITATE temporală $O(|V| \cdot |E|^2)$

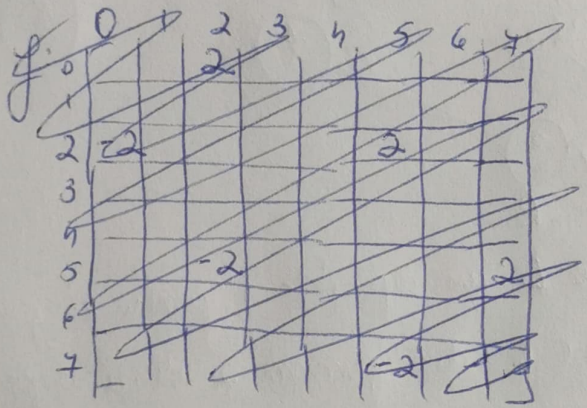
S1.4 2019 (B)



1) BFS(0) cât timp $f[nod][vecin] < c[nod][vecin]$

$p = [0, 2, 5, 7] \Rightarrow c_R(p) = 2$

capacitate muchie: 2 5 2



(39)

2) BFS(0) → $p = [0, 1, 3, 5, 2, 4, 6, 7] \rightarrow c_R(p) = 1$

capacitate	3	2	1	0	1	2	2
flux	0	0	0	-2	0	0	0
	3	2	1	2	1	2	2

3) BFS(0) → $p = []$

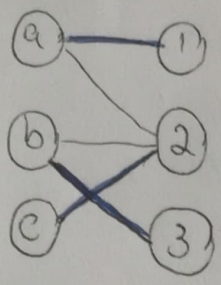
MAXIMUM BIPARTITE MATCHING

Pentru un graf $G = (V, E)$ căutăm submulțimea $M \subseteq E$ a.î.

($\nexists$) $v \in V$ are cel mult o muchie din M incidentă în el

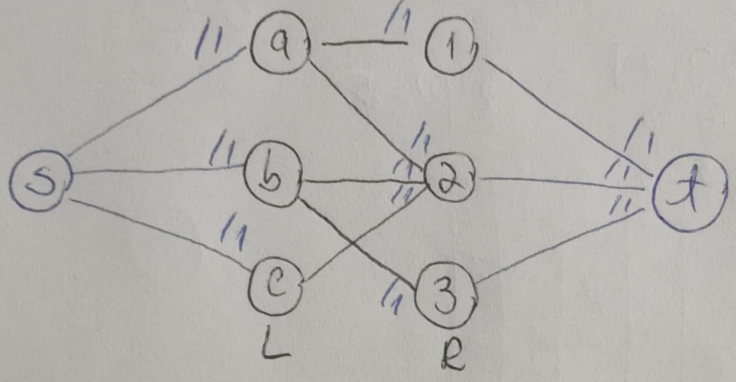
și M are cardinalitate maximă ($\nexists M'$ cu $|M'| \geq |M|$)

Exemplu:



oameni joburi

Transformăm problema într-una de flux:



1) fiecare muchie de la S la L are capacitate 1 pentru a permite un singur match pt. fiecare $v \in L$ (dacă puneam 2 atunci o persoană putea face 2 joburi)
2) fiecare muchie de la R la t are capacitate 1 pentru că există un singur job din fiecare „categorie"/zi (dacă puneam 2 atunci ere ca și cum existau 2 joburi de acel tip și puteau fi luate de 2 persoane)
3) fiecare muchie dintre L și R are capacitatea 1 pentru că o persoană poate lua un singur job la un moment dat (dacă se pot lua mai multe mărim capacitatea de pe muchii până la numărul maxim permis)

ALGORITMI DE EXPLORARE

→ nu pot să construiesc tot spațiul stărilor (graf în care arcele sunt tranziții valide dintr-o stare într-alta)

ALGORITMI
- INFORMAȚI (explorăm pe baza a ce știm deja - informație prelucrată euristic)
- NEINFORMAȚI (random)
- TENTATIVI (pot renunța la calea curentă dacă nu este avantajoasă + revin la o cale anterioară)
- IREVOCABILI (odată ce am ales o cale nu mai pot revenii la restul ⇒ avansez pe o singură direcție)

↓ nu garantează găsirea unei soluții pt. că pierd informații (algoritmi incompleți)

NODURI
- EXPANDATE - toți succesorii sunt cunoscuți ~~multimea~~ CLOSED
- EXPLORATE - știu nodul, dar nu și succesorii — mulțimea OPEN
- NEEXPLORATE - nod necunoscut

(41)

IDEE GENERALĂ: cât timp mai am noduri de prelucrat ($OPEN \neq \emptyset$), scot un nod și îl expandez (îl scot din OPEN și îi adaug succesorii)

1) GRADIENT MAXIM (metoda alpinistului) → ireversibil, nu e optim nici complet
- păstrez în OPEN numai un nod, cel cu valoare maximă
- fiecare nod are asociată valoarea $f(nod)$ - cât de bun e în soluția parțială
- dintre succesorii unui nod îl aleg pe cel cu $f(nod)$ maxim

2) BF* (BEST FIRST)
- $f(n)$ = distanța de la nodul curent la o soluție (căutăm nodurile cu $f(n)$ minim)
- folosim CLOSED pt. a evita ciclurile
  - parent(nod) pt. a ști pe ce tranziție ($p[nod]$, nod) am ajuns la starea curentă (trebuie actualizat atunci când găsim succesori mai bun cu $f(nod)$ mai mic)
- complet, dar neoptim (depinde de euristica f)

```
BF* (stare_initiala, f, test_sol)

    nod = construieste_nod(stare_initiala)
    p[nod] = NULL

    OPEN = [nod] // incep explorare din stare initiala
    CLOSED = ∅

    WHILE OPEN ≠ ∅
        nod = OPEN.pop() // nodul din OPEN cu f(nod) minim
        IF test_sol(nod) // am ajuns la o solutie
            return

        OPEN = OPEN \ {nod}
        CLOSED = CLOSED ∪ {nod}   // marchează nodul ca expandat
        succs = expand(nod)

        FOR succ ∈ succs
            IF succ ∉ OPEN && succ ∉ CLOSED // nod neexplorat
                OPEN = OPEN ∪ {succ}
                p[succ] = nod
                continue

            prev_succ = aparitia anterioară din OPEN ∪ CLOSED
            IF f(prev_succ) < f(succ) // avem o cale mai bună
                p[prev_succ] = nod                    // prin nod
                f[prev_succ] = f[succ]
                IF prev_succ ∈ CLOSED // trebuie să redeschidem
                    CLOSED = CLOSED \ {prev_succ}     // dacă era
                    OPEN = OPEN ∪ {prev_succ}         // închis
                                                      // deja
```

42

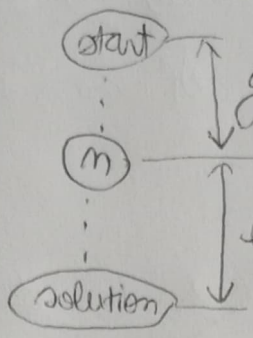
3) A^* → complet, optim dacă euristica este admisibilă

→ varianta BF* dar cu 2 funcții de cost:

$f(n) = g(n) + h(n)$

$f^*(n) = g^*(n) + h^*(n)$

↳ costul exact al unui drum optim



$g(n)$ = costul drumului de la start la n

$g^*(n)$ = costul exact al unui drum optim de la start la n

$h(n)$ = costul estimat de la n la o SOLUȚIE (nu îmi pasă exact care, doar să fie cât mai aproape)

$h^*(n)$ = costul exact —11—

→ CONSTRÂNGERI: $h(n) \geq 0$

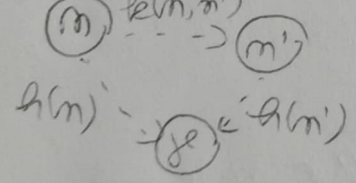
$h(s) = 0$, $s \in S$ = mulțimea soluțiilor

$c[n][n'] > 0$ (costul arcului (n, n'))

→ EURISTICĂ ADMISIBILĂ: $h(n) \leq h^*(n)$, $\forall n$ (OPTIMISTĂ – nu estimează mai mult decât costul exact)

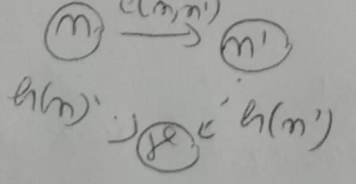
→ —11— CONSISTENȚĂ: $h(n) \leq h(n') + k(n, n')$ și $h(s) = 0$, $\forall s \in S$ soluție

↳ accesibil din n ↳ costul unui drum optim de la n la n'



→ —11— MONOTONĂ: $h(n) \leq h(n') + c(n, n')$

↳ costul arcului



h3

PROPRIETĂȚI

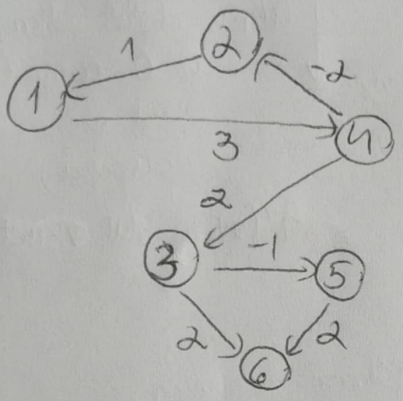
- h consistentă ⇔ h monotonă
- h consistentă ⇒ h admisibilă
- h monotonă ⇒ h admisibilă

h_1, h_2 2 euristici. h_1 este mai informată decât h_2 dacă $h_2(n) < h_1(n), (\forall) n \notin \Gamma$

A_1^* domină A_2^* dacă [orice nod expandat de A_1^* este expandat și de A_2^*
[(echivalent cu) h_1 este mai informată decât h_2

A_1^* condus de h_1 ↑ A_2^* condus de h_2 ↑

S1.5 2020)

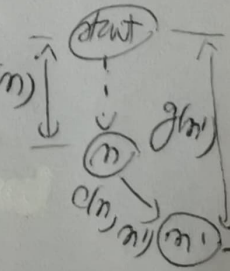


stare inițială = 2
stare finală = 6

nod	1	2	3	4	5	6
h	5	6	1	3	2	0

OPEN = {2, 4}
CLOSED = ∅

```
A*(stare_init, h, test_sol)
    n0 = constr_nod(st_init)
    f(n0) = h(n0), g(n0) = 0, p(n0) = NULL
    OPEN = {n0}
    CLOSED = ∅
    WHILE OPEN ≠ ∅
        nodc = OPEN.pop() // se scoate nodc cu f(nodc) minim
        IF test_sol(nodc) // este soluție
            return nodc
        OPEN = OPEN \ {nodc}
        CLOSED = CLOSED ∪ {nodc}   // marchează nodul ca expandat
        succs = expand(nodc)
        FOR succ ∈ succs
            g_succ = g(nodc) + c(nodc, succ)
            f_succ = g_succ + h(succ)
```



```
IF succ ∉ OPEN∪CLOSED
    OPEN = OPEN ∪ {succ} // the least explored
    p[succ] = node
    g[succ] = g-succ
    f[succ] = f-succ
    continue

IF g-succ < g[succ] // nu actualizez direct g[succ] pentru că
    g[succ] = g-succ // o-ar putea să fie o cale mai proastă decât
    f[succ] = f-succ // cea pe care o am deja
    p[succ] = node
    IF succ ∈ CLOSED
        OPEN = OPEN ∪ {succ}
        CLOSED = CLOSED \ {succ}
```

(45)

Rezolvare la S1.5 2020)

OPEN = {2}, CLOSED = ∅

1) x scoate 2 ⇒ OPEN = ∅, CLOSED = {2}
succs = [1]
succ = 1 ⇒ OPEN = {1}, g(1) = 1
f(1) = 6

2) x scoate 1 ⇒ OPEN = ∅, CLOSED = {2,1}
succs = [4]
succ = 4 ⇒ OPEN = {4}, g(4) = 4
f(4) = 7

	1	2	3	4	5	6
h	5	6	1	3	2	0
g	1	0	6	4	5	~~8~~ 7
f	6	6	7	7	7	~~8~~ 7
p	2	NULL	4	1	3	~~3~~ 5

3) x scoate 4 => OPEN = ∅, CLOSED = {2,1,4}

succs = [2,3]

succ = 2 ∈ CLOSED

g-succ = 2

f-succ = 6

g-succ = 2 > 0 = g[succ] => nu se actualizează g/f pt. 2

succ = 3 => OPEN = {3}, g(3) = 6

f(3) = 7

4) x scoate 3 => OPEN = ∅, CLOSED = {2,1,4,3}

succs = [5,6]

succ = 5 => OPEN = {5}, g(5) = 5

f(5) = 7

succ = 6 => OPEN = {5,6}, g(6) = 8

f(6) = 8

(16)

5) x scoate 5 => OPEN = {6}, CLOSED = {2,1,4,3,5}

succs = [6]

succ = 6 ∈ OPEN

g-succ = 7

f-succ = 7

g-succ = 7 < 8 = g[succ] => g[6] = 7

f[6] = 7

p[6] = 5

6) x scoate 6 => OPEN = ∅, CLOSED = {2,1,4,3,5,6}

succs = []

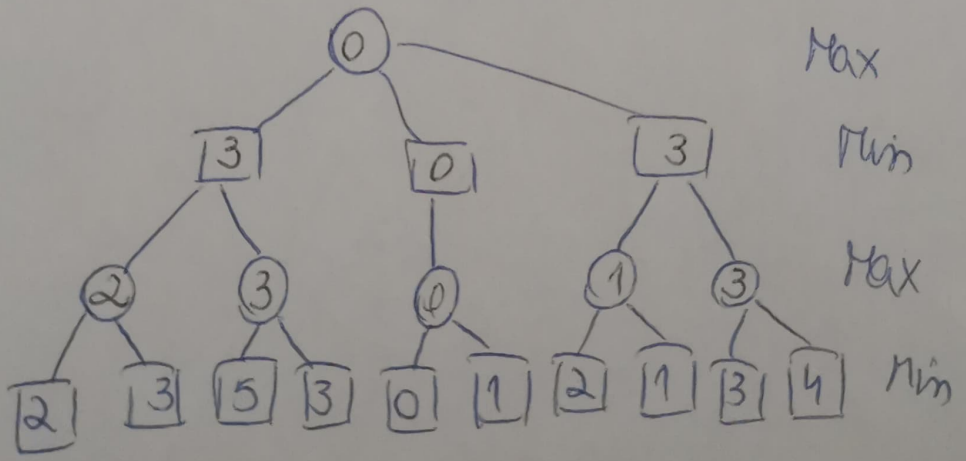
MINIMAX

2 jucători { MIN caută să își minimizeze pierderile (mută pe nivel impar)
MAX caută să își maximizeze câștigurile (mută pe nivel par)

PAȘI

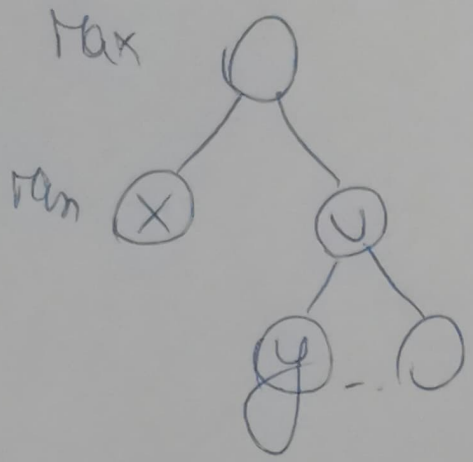
1) Se generează tot arborele
2) Se evaluează frunzele și li se asociază valori (câștig/pierdere cu cât)
3) Se propagă valorile de la frunze spre rădăcină
   - nivel impar (Min) face minimul
   - nivel par (Max) face maximul

S2.2 2017 CB)



minimax + alfa-beta (după)

ALFA-BETA



$y < x \Rightarrow$ Min alege $z \leq y$

$z \leq y < x \Rightarrow$ Max nu alege pe x ci pe y pentru că y pasul fi ignorată

- reduce complexitatea (de la m-ramificări m-nivele la m-ramificări m-nivele/2) ideal
- conceptă valorile în care sunt evaluate nodurile

S2.2 2017(B)

